



HPC
HEIG-VD

Exploration de l'optimisation de code avec des outils LLM

Students:

Kilian Froidevaux
Basile Buxtorf

Teacher:

Alberto Dassatti
Bruno Da Rocha Carvalho

08.06.2026

Table of Contents

Introduction	3
Conception du plugin d'optimisation de code	3
Installation	4
Fonctionnement du workflow	4
Résultats	5
Problème n-body	5
Multiplication de matrices	6
Workflow	6
Simple prompt	6
Discussion	6
Problème n-body	6
Multiplication de matrices	6
Conclusion	7
Bibliography	8

Introduction

Nous avons choisi le second challenge qui est un projet exploratoire sur l'optimisation de code avec des outils LLM.

En nous appuyant sur les résultats de *Lost in the Middle* [1], nous considérons que les informations présentes dans un contexte très long deviennent progressivement plus difficiles à exploiter par les LLM. Les auteurs montrent notamment une baisse des performances lorsque l'information recherchée est située au centre du contexte, ce qui suggère une dispersion de l'attention au sein de contextes étendus.

Nous avons donc décidé d'explorer différentes approches d'optimisation de code en utilisant des outils LLM, pour essayer d'obtenir des meilleurs résultats que ceux obtenus avec un simple prompt.

Nous avons choisi de comparer les résultats obtenus en utilisant:

- Un plugin d'optimisation de code pour Pi.dev, un harness d'agent IA très extensible. Ce plugin utilise des sous-agents spécialisés pour analyser le code, identifier les opportunités d'optimisation, proposer des améliorations et benchmarker les résultats. L'objectif est d'obtenir des gains de performance significatifs avec des modèles moins performants en utilisant moins de tokens. Cela grâce à une approche plus ciblée où chaque sous-agent se concentre sur une tâche spécifique, et n'a accès qu'aux informations nécessaires à sa tâche, ce qui permet de réduire le nombre de tokens utilisés et d'améliorer l'efficacité globale du processus.
- Un simple prompt pour un LLM aussi performant que le plus performant utilisé dans le plugin.

Le but est de comparer et comprendre les avantages et les inconvénients de chaque approche, et d'explorer les possibilités offertes par les outils LLM pour l'optimisation de code.

Conception du plugin d'optimisation de code

La philosophie de Pi.dev est de fournir une plateforme flexible et modulaire qui rend possible n'importe quelle application d'agent IA. La documentation est intégrée dans l'environnement et les agents y ont accès, ce qui permet d'écrire des extensions de manière autonome. L'agent de base reste volontairement minimaliste : plutôt que d'intégrer un grand nombre de fonctionnalités dès le départ, la plateforme lui fournit les outils, les connaissances et l'infrastructure nécessaires pour que des LLM puissent générer et faire évoluer leurs propres modules selon les besoins.

Nous avons choisi cette approche pour écrire le module d'optimisation de code. Cependant, les spécifications ont été écrites manuellement ce qui nous a permis de nous concentrer sur la partie de conception de la pipeline d'optimisation et de l'orchestration plutôt que sur l'implémentation. Les prompts utilisés sont en annexe du rapport.

Installation

Pour installer Pi.dev, il y a plusieurs possibilités, toutes décrites sur leur site web. Il faut ensuite une clé d'API pour pouvoir utiliser un LLM. Nous avons choisi d'utiliser les modèles Kimi de Moonshot AI, qui sont très performants et offrent un bon rapport qualité-prix. Pour obtenir une clé d'API Kimi il faut suivre les instructions à <https://platform.kimi.ai/console/api-keys>.

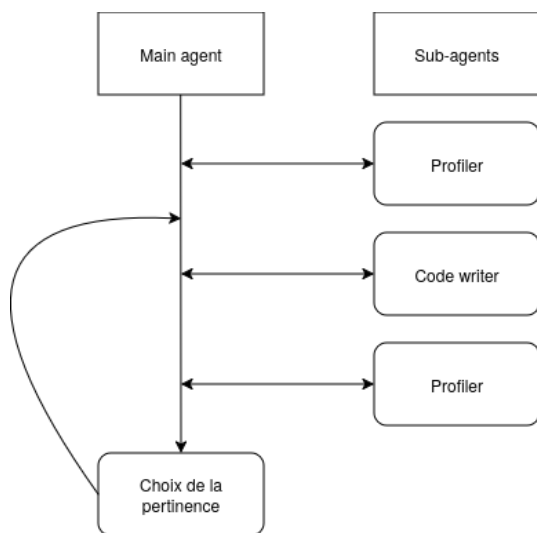
Pour l'accès au modèle, il est aussi possible d'utiliser GitHub Copilot. En l'occurrence, les étudiants de la HEIG-VD ont accès à Copilot gratuitement, ce qui en fait une option intéressante même si avec <https://github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/> [les changements de tarification récents de GitHub Copilot], l'option a perdu de son attrait.

Ensuite, pour installer notre plugin d'optimisation de code, il suffit d'exécuter la commande suivante dans le terminal:

```
pi install git:github.com/BasileBux/hpc-pi-optimizer
```

Pour finir, ouvrir `pi` et utiliser `/hpc-optimize`. Le plugin peut aussi être utilisé dans les cas où on demande à l'agent de faire une optimisation de code, sans avoir besoin d'utiliser la commande spécifique du plugin.

Fonctionnement du workflow



L'agent principal sert à orchestrer les interactions entre les différents sous-agents et à s'assurer que la pipeline reste cohérente.

Il reçoit les résultats de chaque sous-agent, puis prend des décisions basées sur ces résultats pour déterminer les étapes suivantes.

Cette chaîne d'optimisation permet de traiter un aspect à la fois, en se concentrant sur une optimisation spécifique à chaque itération. L'agent principal a une liste de tous les types d'optimisation et les parcourt une par une. Il a ensuite deux sous-agents qu'il peut appeler pour des tâches spécifiques :

1. Profiler: modèle moins performant, spécialisé dans l'exécution de commandes shell pour collecter des métriques de performance et retourner des résultats structurés de manière concise. Il ne fait que cela et n'analyse pas le code ni n'écrit du code.

2. **Optimizer**: modèle plus performant, qui reçoit les résultats structurés du profiler, lit les fichiers sources pertinents, implémente les optimisations, compile, exécute les tests et les benchmarks, le tout en une seule invocation. Il a accès à des outils de lecture, d'écriture, d'édition et de bash. Il préserve la correction du code en comparant les sorties contre les entrées fonctionnelles.

Résultats

Avant même d'obtenir les premiers résultats, lors de la conception de la pipeline d'optimisation, nous avons pu constater que la sur-complexité de la pipeline d'optimisation et le nombre de sous-agents impliqués dans le processus d'optimisation créé un overhead significatif. Cet overhead rend difficile d'obtenir quoi que ce soit. La manière dont fonctionnent les LLM est telle que même en ayant le minimum d'informations nécessaires à leur tâche, ils vont tout de même répéter une partie du travail ce qui rend le processus très lent et aussi coûteux qu'un simple prompt pour un LLM performant.

Notre idée originale était d'utiliser un agent pour chaque sous-tâche (découverte du projet, écriture de tests, recherche d'optimisation, écriture de code et benchmark), mais cela a créé une complexité trop grande qui a rendu le processus inutile. Nous avons donc réduit le nombre d'agents à deux comme décrit dans la section précédente.

TODO:

Document how nbody problem didn't work as it was too hard

Problème n-body

Nous avons tout d'abord essayé d'optimiser un projet de simulation de N-body en C, qui est un projet classique pour tester les optimisations de code. Cependant, aucune des deux approches n'a permis d'obtenir d'optimisation observable.

Nous avons tout d'abord essayé d'optimiser un projet de simulation de N-body en C, qui est un projet classique pour tester les optimisations de code. Cependant, aucune des deux approches n'a permis d'obtenir d'optimisation observable. Plusieurs facteurs expliquent cet échec. Le calcul des forces gravitationnelles, bien qu'il constitue un hotspot évident identifiable par perf, requiert des transformations profondes du code, notamment le passage d'une structure de données array-of-structs (AoS) à struct-of-arrays (SoA) ou l'introduction d'un algorithme de Barnes-Hut, qui impliquent de maintenir une symétrie numérique stricte entre toutes les paires de particules. L'agent LLM, qu'il soit unique ou orchestré, a systématiquement proposé des modifications qui brisaient cette symétrie ou altéraient légèrement les résultats numériques, rendant la vérification de la correction impossible sans intervention manuelle. En d'autres termes, le problème présentait un rapport trop élevé entre la complexité structurelle de l'optimisation et la clarté du signal profilé.

Face à ce constat, nous avons choisi de pivoter vers un second algorithme plus favorable à l'optimisation assistée par IA : la multiplication naïve de matrices carrées. Ce choix

présente deux avantages méthodologiques majeurs. Le hotspot est réduit à une fonction unique contenant trois boucles imbriquées, ce qui limite considérablement l'espace de recherche de l'agent.

Multiplication de matrices

Workflow

Simple prompt

Baseline min-time: 6.505 s Optimized min-time: 129.1 ms Speedup: $\sim 50.4 \times$

Rapport du LLM sur les optimisations proposées:

1. Loop reordering: $i-j-k \rightarrow i-k-j$: perf stat showed 1.3B L1-dcache load misses (37% miss rate) because B was accessed by columns. Reordering makes B row-wise sequential.
2. restrict qualifiers on A, B, C: Eliminates aliasing assumptions; enables the compiler to auto-vectorize with AVX2 (vmulpd/vaddpd) without conservative reloads.
3. aligned_alloc(4096) + pre-fault zeroing: malloc returns 16-byte aligned memory; page alignment eliminates L1 cache-set conflicts between C and B rows. Pre-faulting the zero loop avoids page faults during timing.
4. Pointer arithmetic for rows (`c_row[j] += a * b_row[j]`): Reduces repeated $i*n/k*n$ index arithmetic in the inner loop; compiler generates slightly tighter addressing.
5. Manual $8 \times j$ -unroll & 64×64 blocking: Reverted — both interfered with GCC's auto-vectorization and register allocation, increasing runtime.

Tokens down: 150k

Tokens up: 47k

Cost: 0.877\$

Discussion

Problème n-body

Avec le projet de N-body, nous avons pu constater les limites de l'approche d'optimisation assistée par IA. En effet, lorsque les transformations nécessaires pour optimiser le code sont trop complexes et impliquent des modifications structurelles profondes, les agents LLM ont du mal à proposer des optimisations efficaces tout en préservant un code correct. De plus, le signal profilé est souvent noyé dans le bruit des modifications proposées, ce qui rend difficile pour les agents de faire des choix éclairés. Dans ce cas, l'approche d'optimisation assistée par IA ne semble pas être la plus adaptée, et une intervention humaine est nécessaire pour l'optimisation.

Multiplication de matrices

Nous avons pu constater que la pipeline d'optimisation avec les sous-agents spécialisés permet d'obtenir des résultats plus convaincants. La performance obtenue est meilleure que celle obtenue avec un simple prompt. En effet, le fait d'avoir des sous-agents spécialisés permet de se concentrer sur des aspects spécifiques de l'optimisation, ce qui permet d'obtenir des résultats plus ciblés et plus efficaces. De plus, il est beaucoup plus

facile de contrôler le nombre d'itérations et les modèles ont moins de chances de se perdre car leur contexte est plus limité.

Les résultats obtenus sont cohérents avec les observations rapportées dans *Lost in the Middle* [1]. En effet, la séparation du contexte entre plusieurs sous-agents spécialisés permet de limiter la quantité d'informations que chaque modèle doit traiter simultanément. Cette approche réduit les effets liés aux contextes très longs mis en évidence par les auteurs et conduit, dans notre cas, à des optimisations de meilleure qualité. Nos expérimentations tendent ainsi à confirmer l'hypothèse selon laquelle une réduction et une spécialisation du contexte améliorent l'exploitation des informations pertinentes par les LLM.

TODO:

Add more details on the results obtained and maybe some limitations and advantages of each approach

Nous avons pu constater que la pipeline d'optimisation avec les sous-agents spécialisés ne permet pas d'obtenir des résultats convaincants. La performance obtenue est moins bonne que celle obtenue avec un simple prompt. En effet, le fait d'avoir des sous-agents spécialisés crée un overhead significatif et leur contexte est trop séparé, ce qui rend le processus complexe et inefficace.

La grande faiblesse des LLM qui est que leur contexte est dilué avec le temps est aussi une force car elle leur permet de donner de meilleurs résultats en ayant une vue d'ensemble du projet et de l'optimisation à faire.

Cependant, avec nos moyens limités, nous n'avons pas effectué nos tests sur un projet suffisamment grand pour que le problème de dilution du contexte soit marqué jusqu'aux résultats obtenus avec un simple prompt. Il est possible que sur un projet plus grand et plus complexe, la pipeline d'optimisation avec les sous-agents spécialisés puisse obtenir de meilleurs résultats que le simple prompt.

Note: les LLM sont de nature non déterministes, ce qui rend le processus de test très difficile et aléatoire. Par manque de temps et de ressources, nous n'avons pas pu effectuer suffisamment de tests pour obtenir des résultats concluants

Conclusion

Bibliography

- [1] N. F. Liu *et al.*, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.